

CS 352

Computer Graphics

Lecture 4

Design of Line Algorithms

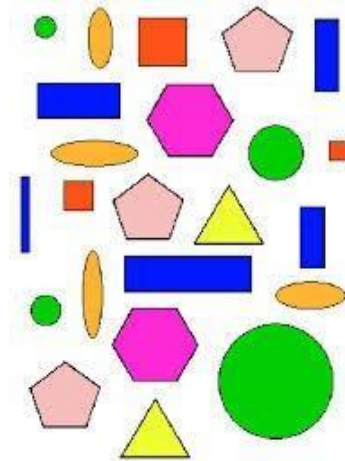


Output primitives

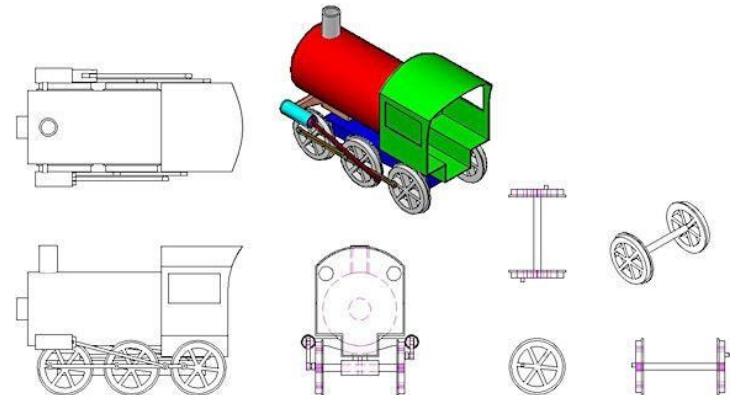
- Graphics programming packages provide functions to describe a scene in terms of basic geometric structures referred to as Output Primitives

- Output primitives include:**

- Points
- Lines
- Circles
- Spline curves



- Output primitives can be used to present more complex objects**



Line Drawing Algorithms

A) Parametric Form Of The Equation Of A 2D Line Segment

Given points $P_1 = (x_1, y_1)$ and $P_2 = (x_2, y_2)$

$$x = x_1 + t(x_2 - x_1)$$

$$y = y_1 + t(y_2 - y_1)$$

t is called the parameter. When

$t = 0$ we get (x_1, y_1)

$t = 1$ we get (x_2, y_2)

As $0 < t < 1$ we get all the other points on the line segment between (x_1, y_1) and (x_2, y_2) .



Line Drawing Algorithms

The challenge is to find a way to calculate the next x,y position by previous one as quickly as possible.



B) Direct Scan conversion Line Algorithm

Slope-Intercept Formula For A Line

Given a third point on the line:

$$P = (x, y)$$

$$\begin{aligned}\text{Slope} &= (y - y_1)/(x - x_1) \\ &= (y_2 - y_1)/(x_2 - x_1)\end{aligned}$$

Solving For y

$$\begin{aligned}y - y_1 &= [(y_2 - y_1)/(x_2 - x_1)](x - x_1) \\ y &= [(y_2 - y_1)/(x_2 - x_1)]x + y_1 - [(y_2 - y_1)/(x_2 - x_1)]x_1\end{aligned}$$

$$y = [(y_2 - y_1)/(x_2 - x_1)]x + y_1 - [(y_2 - y_1)/(x_2 - x_1)]x_1$$

$$y = Mx + B$$

therefore

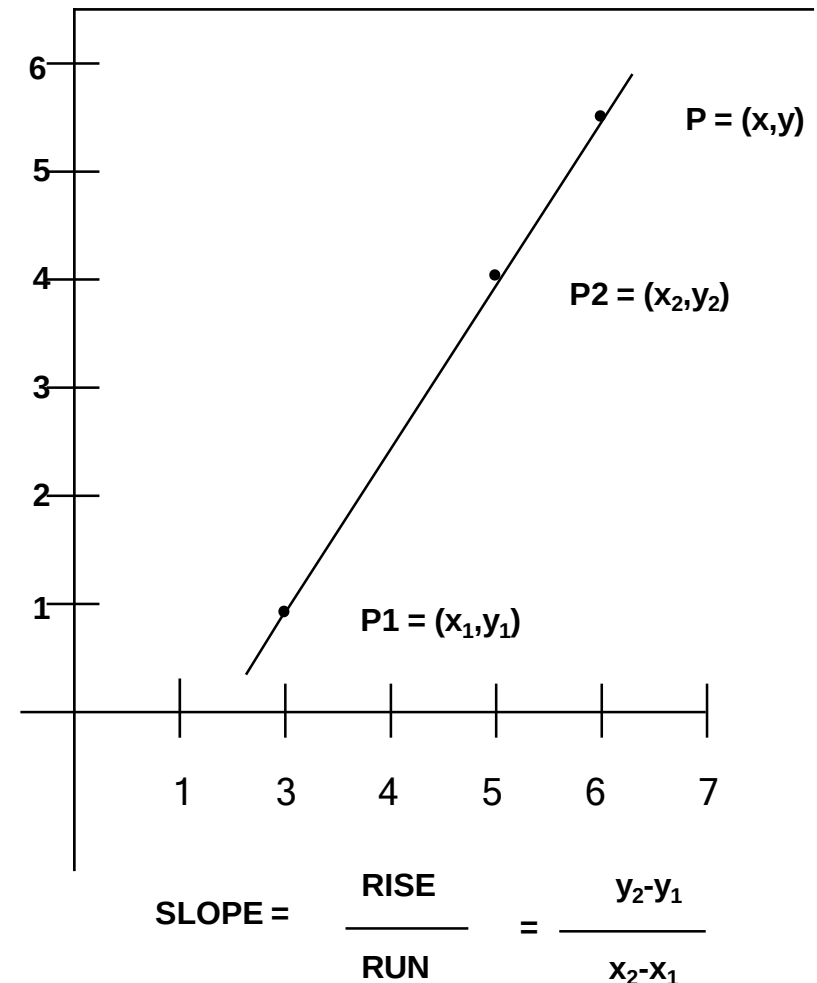
$$y = Mx + B$$

where

$$M = [(y_2 - y_1)/(x_2 - x_1)]$$

$$B = [-(y_2 - y_1)/(x_2 - x_1)]x_1 + y_1$$

Cartesian Coordinate System





B) Direct Scan conversion Line Algorithm

- Input the two end points (x_1, y_1) and (x_2, y_2) .
- Compute the slope of the line, $m = (y_2 - y_1) / (x_2 - x_1)$
- Compute the intercept of the line, $b = y_1 - m * x_1$
- Set initial value of x and y as $x = x_1$, $y = y_1$
- Plot the first end point, `plotpixel(x,y)`
- If $m \leq 1$ then
 - while ($x \leq x_2$)
 - $x = x + 1$;
 - $y = m * x + b$; round off y;
 - `plotpixel(x,y)`;
 - end while
- Else
 - while ($y \leq y_2$)
 - $y = y + 1$;
 - $x = (y - b) / m$; round off x;
 - `plotpixel(x,y)`
 - end while
- End if



B) **Example1:** Direct Scan conversion Line Algorithm

- Show in steps how to draw a line between the two points: $(x_1, y_1) = (3, 4)$ and $(x_2, y_2) = (9, 5)$.
- **Solution**
- Compute the slope of the line, $m = (y_2 - y_1) / (x_2 - x_1) = (5 - 4) / (9 - 3) = 1/6 < 1$
- Compute the intercept of the line, $b = y_1 - m * x_1 = 4 - (1/6) * 3 = 7/2$
- $m \leq 1$: $x = x + 1$; $y = m * x + b$; round off y
- $m > 1$: $y = y + 1$; $x = (y - b) / m$; round off x ;

x	y
3	4
4	$4.17 = 4$
5	$4.33 = 4$
6	$4.50 = 5$
7	$4.66 = 5$
8	$4.83 = 5$
9	5



C- DDA Line Algorithm

(digital differential analyzer)

- The digital differential analyser (DDA) is a scan-conversion line algorithm based on using ΔX or ΔY .
- A line is sampled at unit intervals in one coordinate and the corresponding integer values nearest the line path are determined for the other coordinate.

◆ Advantages:

- 1) **Faster than using $y = Mx + c$**
- 2) **Eliminates multiplication (mx) by using**
screen characteristics, i.e. incrementing pixel

◆ Disadvantages:

- 1) **Floating point arithmetic is slower than** integer arithmetic
- 2) **Rounding down take time**
- 3) **Long lines can drift from the true line due to floating point round-off error**



C- DDA Line Algorithm

- **Step-01:**

Calculate ΔX , ΔY and M from the given input.

- **Step-02:**

Calculate the number of steps.

If $|\Delta X| < |\Delta Y|$, so number of steps = ΔY

If $|\Delta X| > |\Delta Y|$, so number of steps = ΔX

If $|\Delta X| = |\Delta Y|$, so number of steps = ΔY

- **Step-03:**

If $M < 1$, so increment X , and add M to Y .

If $M = 1$, so increment X , and Increment Y .

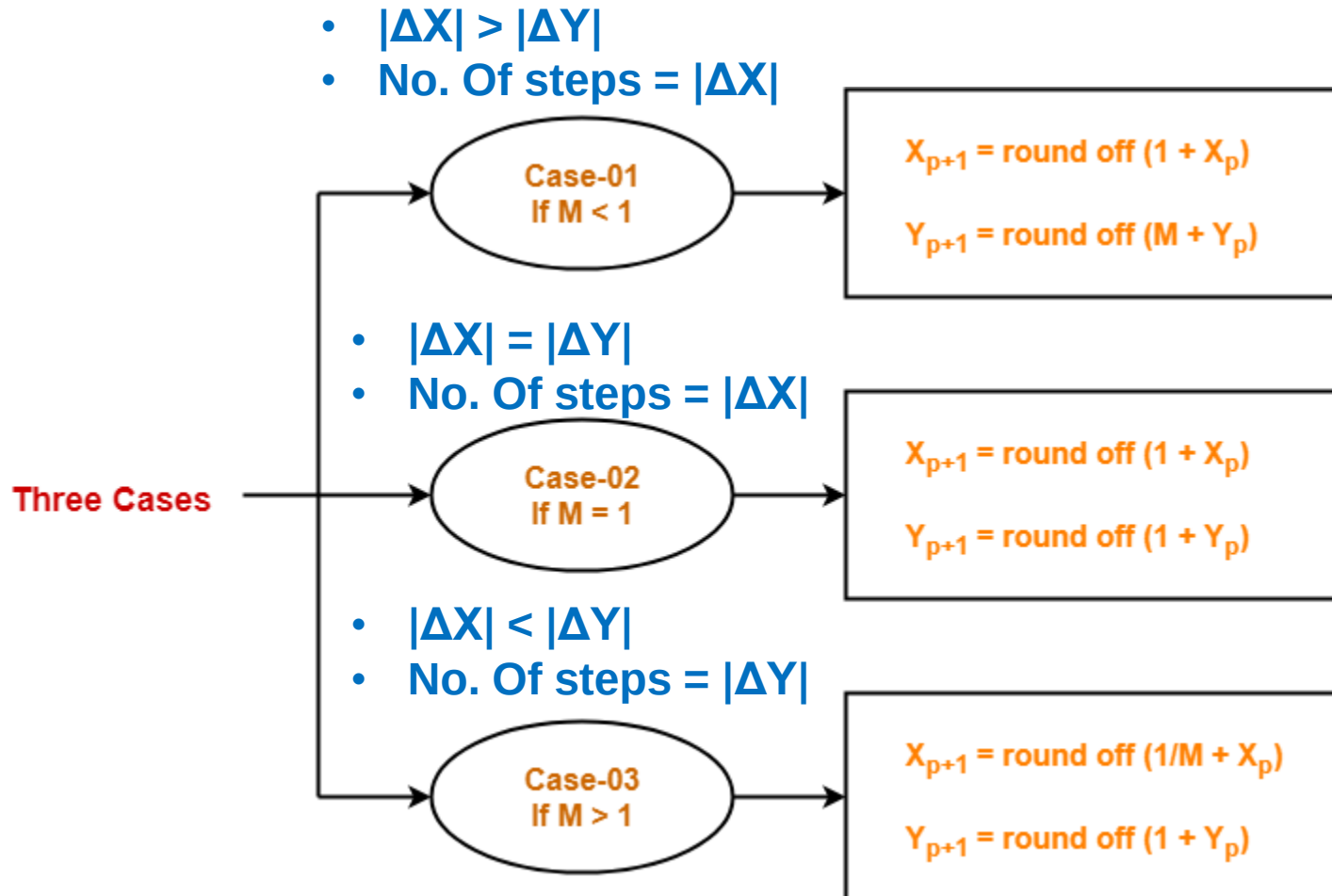
If $M > 1$, so Add $1/m$ to X , and add 1 to Y .

- **Step-04:**

Repeat Step-03 until the end point is reached or the number of generated new points equals to the number of steps.



C- DDA Line Algorithm





C- DDA Line Algorithm

Example1: Calculate the points between the starting point (5, 6) and ending point (8, 12).

- **Step-1:**

Calculate ΔX , ΔY and M from the given input.

$$\Delta X = X_2 - X_1 = 8 - 5 = 3$$

$$\Delta Y = Y_2 - Y_1 = 12 - 6 = 6$$

$$M = \Delta Y / \Delta X = 6 / 3 = 2$$

- **Step-2:**

Calculate the number of steps.

As $|\Delta X| < |\Delta Y| = 3 < 6$, so number of steps = $\Delta Y = 6$

- **Step-3:**

As $M > 1$, so Add $1/m$ to X , and add 1 to Y .

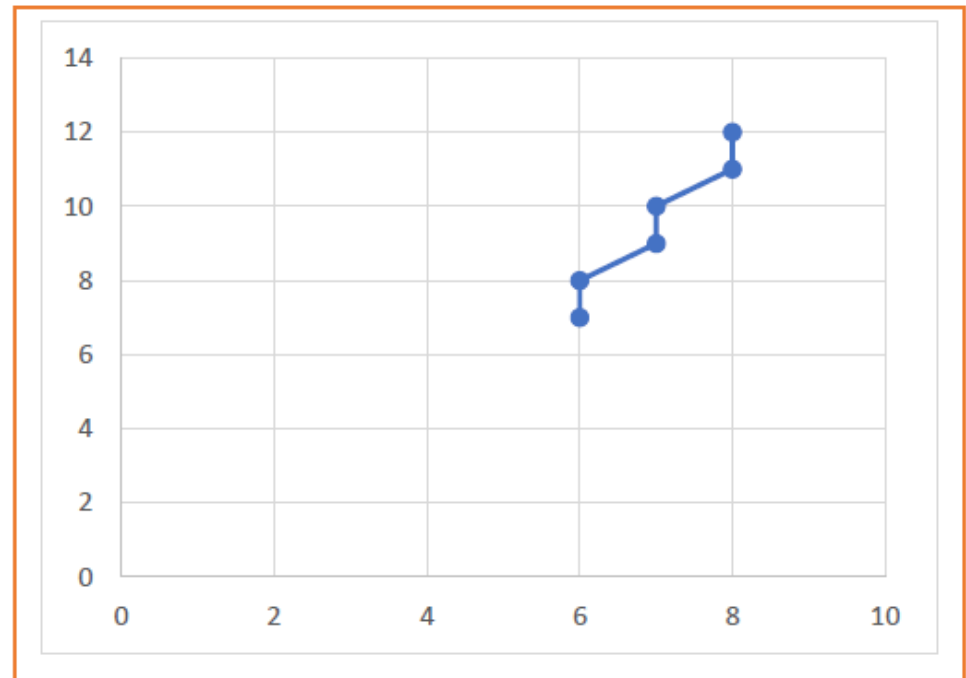
- **Step-4:**

Repeat Step-3 until reach to the end point.



C- DDA Line Algorithm

X_p	Y_p	X_{p+1}	Y_{p+1}	Round off (X_{p+1}, Y_{p+1})
5	6	5.5	7	(6, 7)
		6	8	(6, 8)
		6.5	9	(7, 9)
		7	10	(7, 10)
		7.5	11	(8, 11)
		8	12	(8, 12)





C- DDA Line Algorithm

Example2: Calculate the points between the starting point (1, 7) and ending point (11, 17)..

- **Step-1:**

Calculate ΔX , ΔY and M from the given input.

$$\Delta X = X_n - X_0 = 11 - 1 = 10$$

$$\Delta Y = Y_n - Y_0 = 17 - 7 = 10$$

$$M = \Delta Y / \Delta X = 10 / 10 = 1$$

- **Step-2:**

Calculate the number of steps.

As $|\Delta X| = |\Delta Y| = 10 = 10$, so number of steps = $\Delta X = \Delta Y = 10$

- **Step-3:**

As $M = 1$, so increment X , and Increment Y .

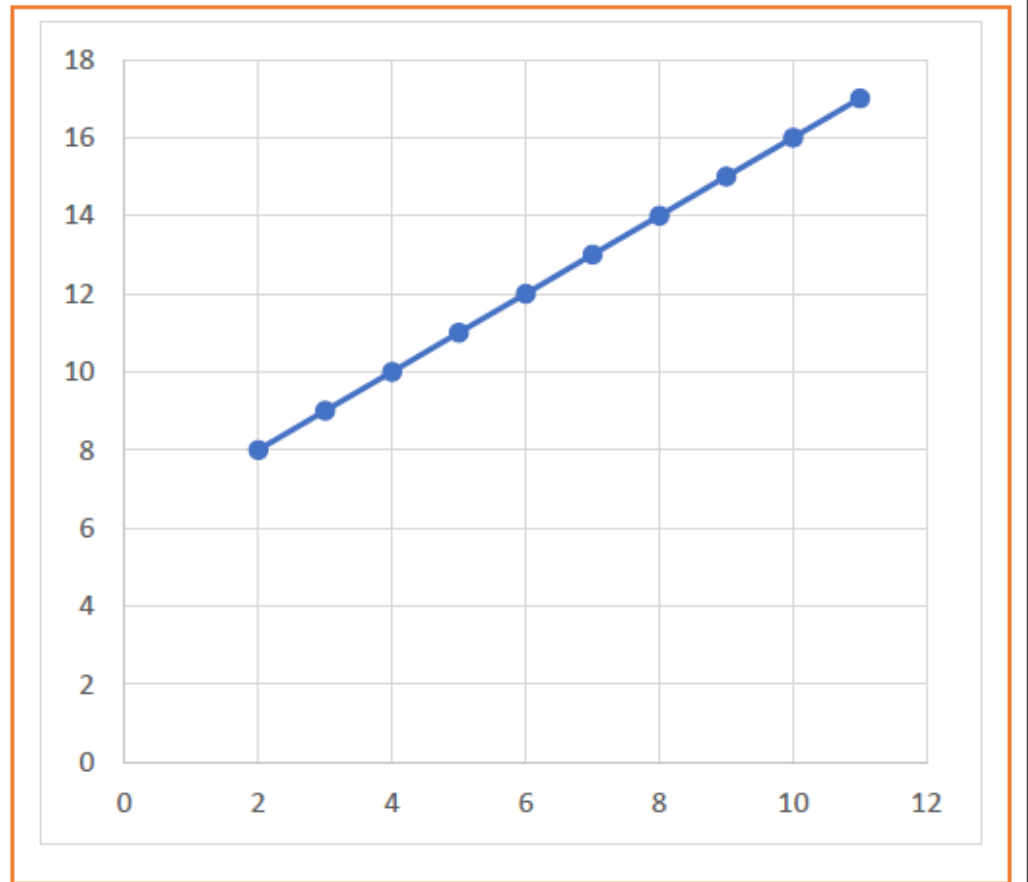
- **Step-4:**

Repeat Step-3 until reach to the end point.



C- DDA Line Algorithm

X_p	Y_p	X_{p+1}	Y_{p+1}	Round off (X_{p+1}, Y_{p+1})
1	7	2	8	(2, 8)
		3	9	(3, 9)
		4	10	(4, 10)
		5	11	(5, 11)
		6	12	(6, 12)
		7	13	(7, 13)
		8	14	(8, 14)
		9	15	(9, 15)
		10	16	(10, 16)
		11	17	(11, 17)





C- DDA Line Algorithm

Example3: Calculate the points between the starting point (5, 6) and ending point (13, 10)..

- **Step-1:**

Calculate ΔX , ΔY and M from the given input.

$$\Delta X = X_n - X_0 = 13 - 5 = 8$$

$$\Delta Y = Y_n - Y_0 = 10 - 6 = 4$$

$$M = \Delta Y / \Delta X = 4 / 8 = 0.50$$

- **Step-2:**

Calculate the number of steps.

As $|\Delta X| > |\Delta Y| = 8 > 4$, so number of steps = $\Delta X = 8$

- **Step-3:**

As $M < 1$, so increment X , and add M to Y .

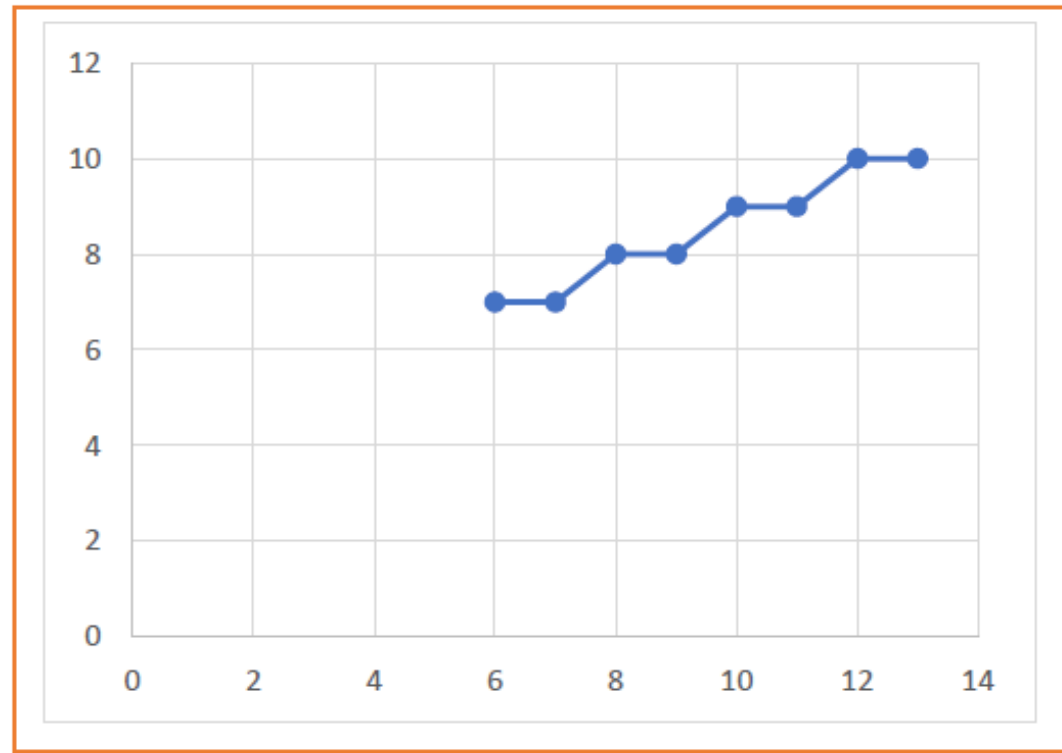
- **Step-4:**

Repeat Step-3 until reach to the end point.



C- DDA Line Algorithm

X_p	Y_p	X_{p+1}	Y_{p+1}	Round off (X_{p+1}, Y_{p+1})
5	6	6	6.5	(6, 7)
		7	7	(7, 7)
		8	7.5	(8, 8)
		9	8	(9, 8)
		10	8.5	(10, 9)
		11	9	(11, 9)
		12	9.5	(12, 10)
		13	10	(13, 10)



C- DDA Line Algorithm

```
inline int round (const float a) { return int (a + 0.5); }
```

```
void lineDDA (int x0, int y0, int xEnd, int yEnd)  
{
```

```
    int dx = xEnd - x0, dy = yEnd - y0, steps, k;  
    float xIncrement, yIncrement, x = x0, y = y0;
```

```
    if (fabs (dx) > fabs (dy))
```

```
        steps = fabs (dx);
```

```
    else
```

```
        steps = fabs (dy);
```

```
    xIncrement = float (dx) / float (steps);
```

```
    yIncrement = float (dy) / float (steps);
```

```
    setPixel (round (x), round (y));
```

```
    for (k = 0; k < steps; k++) {
```

```
        x += xIncrement;
```

```
        y += yIncrement;
```

```
        setPixel (round (x), round (y));
```

```
    }
```

```
}
```

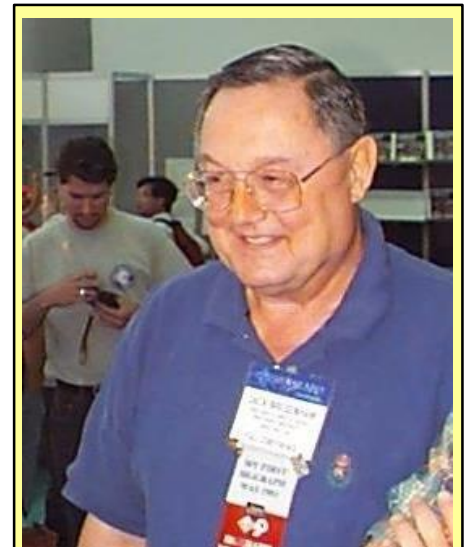
C- DDA Line Algorithm

- Need a lot of floating point arithmetic.
 - 2 'round's and 2 adds per pixel.
- Is there a simpler way ?
- Can we use only integer arithmetic ?
 - Easier to implement in hardware.



D- Bresenham's Line Algorithm

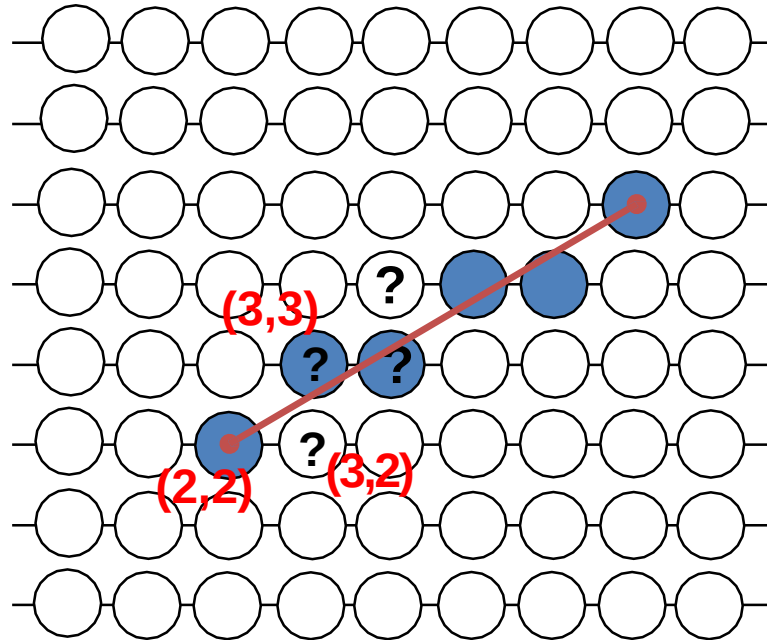
- The Bresenham algorithm is another incremental scan conversion algorithm
- The big advantage of this algorithm is that
 - it uses only integer calculations
 - good for hardware implementation
 - good for assembly language
 - Accurate and efficient
 - Can be adapted to draw circles and curves



Jack Bresenham worked for 27 years at IBM before entering academia. Bresenham developed his famous algorithms at IBM in the early 1960's



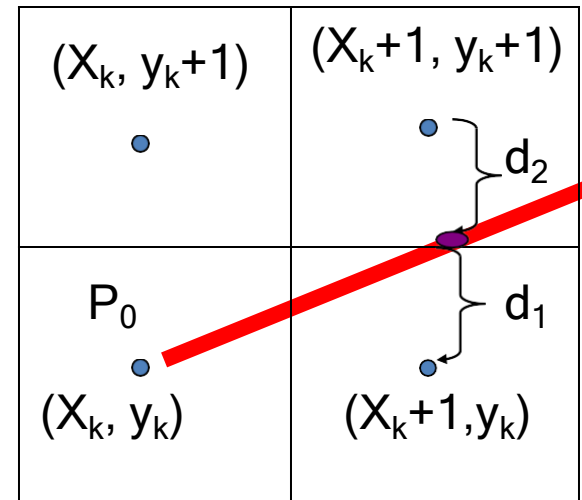
D- Bresenham's Line Algorithm



- **The next sample positions can be plotted either at (3,2) or (3,3) ?**
 - If the pixel to be displayed occurs at a position (X_k, Y_k) then the next pixel is either at (X_k+1, Y_k) or (X_k+1, Y_k+1) i.e, (3,2) or (3,3)

D- Bresenham's Line Algorithm

- The 'Y' coordinate at the pixel position X_{k+1} can be obtained from : $Y = m(X_k + 1) + b$
- the separation between (X_{k+1}, Y_k) and (X_{k+1}, Y) is d_1 and the separation between (X_{k+1}, Y) and (X_{k+1}, Y_{k+1}) is d_2 then



- $d_1 = y - y_k = m(x_k + 1) + b - Y_k$
- $d_2 = (Y_{k+1}) - Y = (Y_{k+1}) - [m(X_k + 1) + b] = (Y_{k+1}) - m(X_k + 1) - b$
- The difference is given as : $d_1 - d_2 =$

$$= m(X_k + 1) + b - Y_k - [(Y_{k+1}) - m(X_k + 1) - b]$$

$$= m(X_k + 1) + b - Y_k - (Y_{k+1}) + m(X_k + 1) + b$$

$$d_1 - d_2 = 2m(X_k + 1) - 2Y_{k+1} + 2b - 1$$



D- Bresenham's Line Algorithm

- A decision parameter P_k can be obtained by substituting $m = dy/dx$ in

$$d1-d2 = 2m(X_k+1)-2Y_k+2b-1$$

$$\begin{aligned} d1-d2 &= 2m(X_k+1)-2Y_k+2b-1 \\ &= 2 \frac{dy}{dx} (X_k+1) - 2Y_k + 2b - 1 \\ &= \frac{2 dy(X_k+1) - 2 dx.Y_k + 2b.dx - dx}{dx} \end{aligned}$$

$$\begin{aligned} dx(d1-d2) &= 2 dy(X_k+1) - 2 dx.Y_k + 2b.dx - dx \\ &= 2 dyX_k + 2 dy - 2 dx.Y_k + 2b.dx - dx \\ &= 2 dyX_k - 2 dx.Y_k + c \end{aligned}$$

Where, $dx(d1-d2) = P_k$ and

$$c = 2 dy + dx(2b-1)$$

$$P_k = 2 dyX_k - 2 dx.Y_k + c \quad (1)$$

- The value of c is constant and is independent of the pixel position. It can be deleted in the recursive calculations, or for P_k
- if $d1 < d2$ (i.e, Y_k is nearer to the line path than Y_{k+1}) then, P_k is negative.
- If P_k is -ve, a lower pixel (Y_k) is plotted else, an upper pixel (Y_{k+1}) is plotted.
- At $k+1$ step, the value of P_k is given as
- $P_{k+1} = 2 dyX_{k+1} - 2 dx.Y_{k+1} + c \quad (2)$



D- Bresenham's Line Algorithm

- Eq 1 – eq 2
- $P_{k+1} - P_k = (2 dy X_{k+1} - 2 dx Y_{k+1} + c) - (2 dy X_k + 2 dx Y_k + c)$
 $= 2dy(X_{k+1} - X_k) - 2 dx(Y_{k+1} - Y_k) \dots\dots\dots(3)$
- Since $X_{k+1} = X_k + 1$

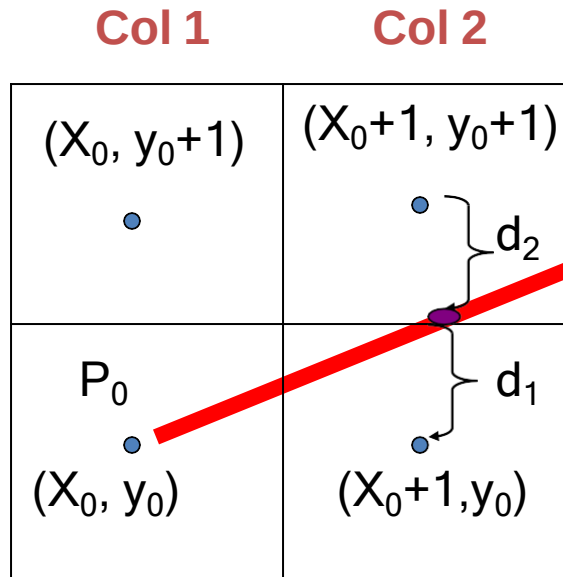
$$P_{k+1} - P_k = 2dy(X_k + 1 - X_k) - 2 dx(Y_{k+1} - Y_k)$$
$$= 2dy - 2 dx(Y_{k+1} - Y_k)$$

$$P_{k+1} = P_k + 2dy - 2 dx(Y_{k+1} - Y_k) \dots\dots\dots(4)$$

Where $(Y_{k+1} - Y_k)$ is either 0 or 1 based on the sign of P_k .

- The starting parameter P_0 at the pixel position (X_0, Y_0) is given as
- $P_0 = 2dy - dx \dots\dots\dots(5)$

D- Bresenham's Line Algorithm



$y=mx+c$ is the eq of line

In col 2 the line is passing through x_0+1 so the y value is given by

$$y=m(x_0+1)+c$$

Now we need to find out the values of $d1$ and $d2$

$$d1 = y - y_0$$

$$d2 = (y_0+1) - y$$

$$d1 = m(x_0+1) - y_0 \text{ and } d2 = (y_0+1) - m(x_0+1)$$

$$d1-d2 = [mx_0+m - y_0] - [(y_0+1) - mx_0-m]$$

$$= mx_0+m - y_0 - y_0 - 1 + mx_0+m$$

$$= 2mx_0+2m-2y_0-1$$

$$= 2mx_0+2m-2mx_0-1 \quad (y=mx+c \text{ passes thr}$$

(x_0, y_0) so we can say $y_0=mx_0+c$)

$$d1-d2 = \cancel{2mx_0}+2m-\cancel{2mx_0}-1$$

$$d1-d2 = 2m-1 \quad (m = dy / dx)$$

$$d1-d2 = 2dy/dx - 1$$

$$dx(d1-d2) = 2dy - dx$$

$$P_0 = 2dy - dx$$



Bresenham's algorithm

- **Step 1:** Enter the 2 end points for a line and store the left end point in (X_0, Y_0) .
- **Step 2:** Plot the first point by loading (X_0, Y_0) in the frame buffer.
- **Step 3:** determine the initial value of the decision parameter by calculating the constants dx , dy , $2dy$ and $2dy-2dx$ as
 - $P_0 = 2dy - dx$
- **Step 4:** for each X_k , conduct the following test, starting from $k=0$
 - If $P_k < 0$, then the next point to be plotted is at (X_{k+1}, Y_k) and
 - $P_{k+1} = P_k + 2dy$
 - Else, the next point is (X_{k+1}, Y_{k+1}) and
 - $P_{k+1} = P_k + 2dy - 2dx$ (step 3)
- **Step 5:** iterate through step (4) dx times.



Bresenham's algorithm

```
Void LineBres( int x1, int y1, int x2, int y2)
{
    int dx =abs(x2-x1), dy= abs(y2-y1)
    int p=2* dy- dx ;
    int x, y, Xend;    // Xends is similar to steps in DDA
    If(x1>x2)
    {
        x=x2;y=y2;Xend=x1;
    }
    else
    {
        x=x1;y=y1;Xend=x2;
    }
    setPexel( x, y );
```

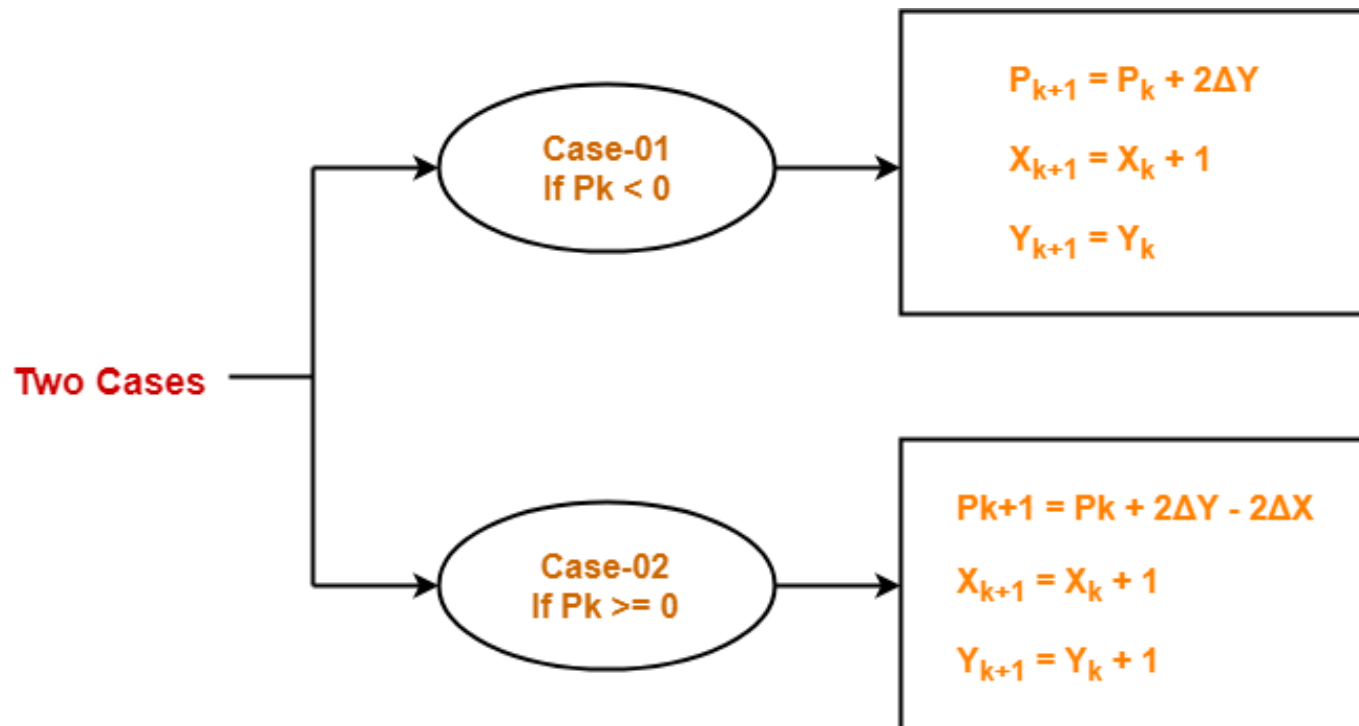
```
While( x < Xend)
{
    x++;
    if( p < 0)
        p=p+2dy;
    else
    {
        y++;
        p=p+2dy-2dx;
    }
    setPixel(x,y);
} // End While

} //End LineBres
```



Advantages of Bresenham's Alg.

- It uses only integer calculations
- So, it is faster than DDA



Example 1

- Let the given end points for the line be (30,20) and (40, 28)
- $M = \frac{dy}{dx} = \frac{y_2 - y_1}{x_2 - x_1} = \frac{28 - 20}{40 - 30} = \frac{8}{10}$
- $m = 0.8$
- $dy = 8$ and $dx = 10$
- The initial decision parameter P_0 is
- $P_0 = 2dy - dx = 2(8) - 10 = 16 - 10 = 6$
- $P_0 = 6$
- The constants $2dy$ and $2dy-2dx$ are
- $2dy = 2(8) = 16$ $2dy-2dx = 2(8) - 2(10) = 16 - 20 = -4$
- $2dy = 16$ $2dy - 2dx = -4$



Example 1

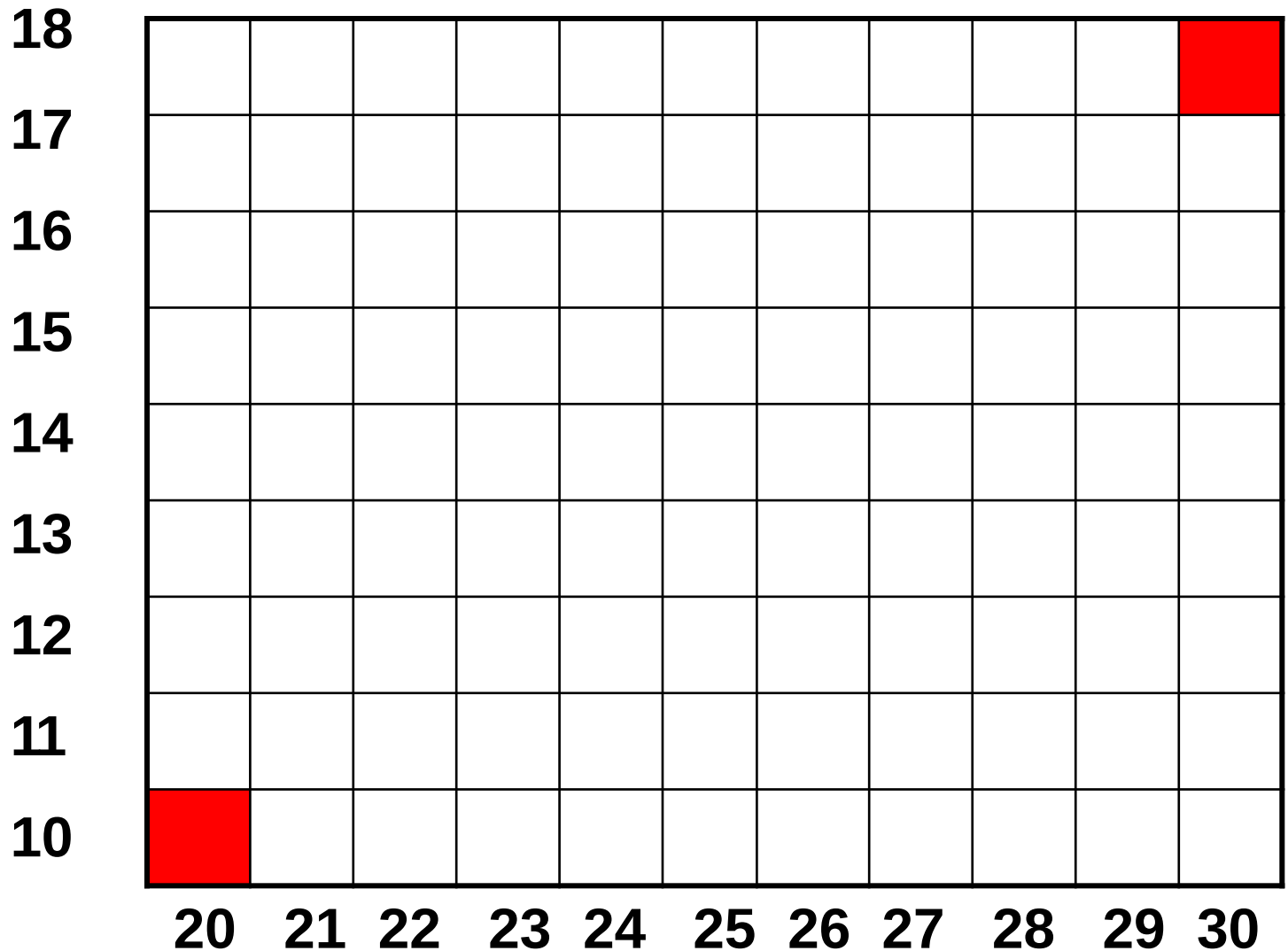
- End points (30,20) (40,28)
- $dx=x_2-x_1=40-30=10$ $dy=y_2-y_1=28-20=8$
- $P_0=2dy-dx=2 \times 8-10 = 6 > 0$ pt(31,21)
- $P_{k+1}= p_k+2dy-2dx= 6+16-20 = 2 > 0$ (32,22)
- $P_{k+1}=2+16-20 = -2 < 0$ (33,22)
- $P_{k+1}= p_k+2dy = -2+16 = 14 > 0$ (34,23)
- $P_{k+1}= p_k+2dy-2dx=14+16-20=10 > 0$ (35,24)
- $P_{k+1}=10+16-20=6 > 0$ (36,25)
- $P_{k+1}= 6+16-20=2 > 0$ (37,26)
- $P_{k+1}=2+16-20=-2 < 0$ (38,26)
- $P_{k+1}= p_k+2dy=-2+16=14 > 0$ (39,27)
- $P_{k+1}=p_k+2dy-2dx= 14+16-20=10 > 0$ (40,28)

The starting point $(x_0, y_0)=(30,20)$ and the successive pixel positions are given in the following table

K	P_k	(X_{k+1}, Y_{k+1})
0	6	(31,21)
1	2	(32,22
2	-2	33,22
3	14	34,23
4	10	35,24
5	6	36,25
6	2	37,26
7	-2	38,26
8	14	39,27
9	10	40,28

Bresenham's line algorithm

Example 2: a line from (20, 10) to (30, 18)



Bresenham's line algorithm

$$\Delta x = (30 - 20) = 10$$

$$\Delta y = (18 - 10) = 8$$

Slope of the line:

$$m = \frac{\Delta y}{\Delta x} = \frac{8}{10} = 0.8$$

$$p_0 = 2\Delta y - \Delta x = 6$$

$$p_k > 0$$

$$(x_k + 1, y_k + 1)$$



$$(21, 11)$$

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

$$2\Delta y - 2\Delta x = -4$$



$$2$$

$$p_k > 0$$

$$(x_k + 1, y_k + 1)$$



$$(22, 12)$$

$$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

$$2\Delta y - 2\Delta x = -4$$



$$-2$$

$$p_k < 0$$

$$(x_k + 1, y_k)$$



$$(23, 12)$$

$$p_{k+1} = p_k + 2\Delta y$$

$$2\Delta y = 16$$

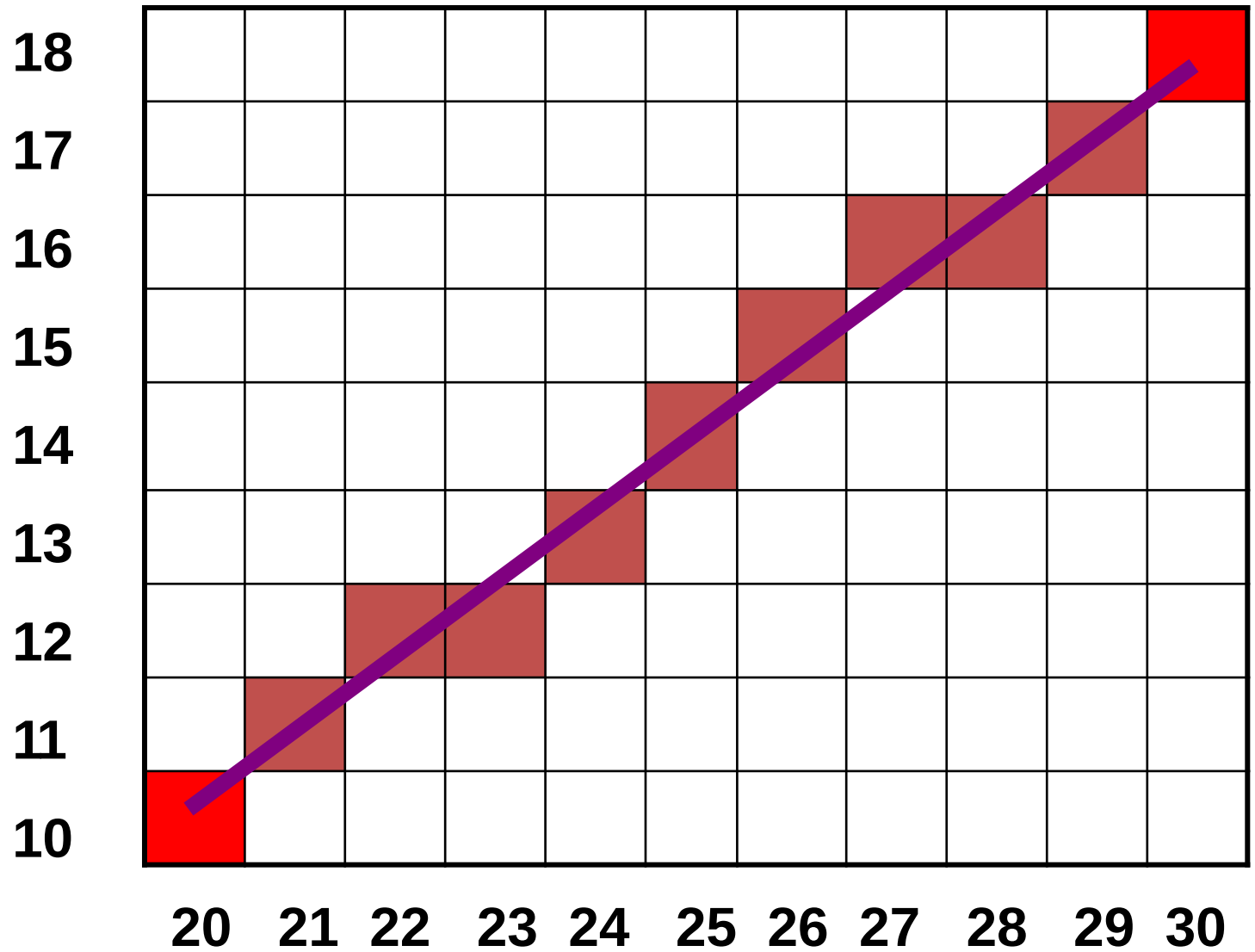


$$14$$

Bresenham's line algorithm

k	p(k)	X(k+1), y(k+1)
0	6	(21,11)
1	2	(22,12)
2	-2	(23,12)
3	14	(24,13)
4	10	(25,14)
5	6	(26,15)
6	2	(27,16)
7	-2	(28,16)
8	14	(29,17)
9	10	(30,18)

Bresenham's line algorithm





Bresenham's algorithm: special cases

- In bresenham's algorithm, if the positive slope of a line is greater than 1, the roles of x and y are interchanged.
- **For positive slope lines:**
 1. If the initial position of a line is the right end point, then both x and y are decremented as we move from right to left.
 2. If $d1 = d2$ then always select the upper or the lower candidate pixel.
- **For negative slope lines:**
 - One coordinate increases and the other coordinate decreases.
- **Special cases:**
 - the vertical lines $dx = 0$, horizontal lines $dy = 0$ and diagonal lines $|dx| = |dy|$ can be directly loaded into the frame buffer.

Bresenham's algorithm: special cases

Example 1: Drawing a Horizontal Line

Let's draw a horizontal line from (2, 3) to (8, 3).

Solution

Step 1: Identify the special case

In this case, $dy = 0$, indicating a horizontal line.

Step 2: Iterate and plot points

- Start at (2, 3)
- Increment x from 2 to 8, keeping y constant at 3.
- Plot the points (3, 3), (4, 3), (5, 3), (6, 3), (7, 3), and (8, 3).

Bresenham's algorithm: special cases

Example 2: Drawing a Vertical Line

Let's draw a vertical line from (5, 1) to (5, 6).

Solution

Step 1: Identify the special case

In this case, $dx = 0$, indicating a vertical line.

Step 2: Iterate and plot points

- Start at (5, 1)
- Increment y from 1 to 6, keeping x constant at 5.
- Plot the points (5, 2), (5, 3), (5, 4), (5, 5), and (5, 6).

Bresenham's algorithm: special cases

Example 3: Drawing a Diagonal Line

Let's draw a diagonal line from (1, 1) to (5, 5).

Solution

Step 1: Identify the special case

In this case, $|dx| = |dy|$, indicating a diagonal line.

Step 2: Iterate and plot points

- Start at (1, 1)
- Increment both x and y in each step.
- Plot the points (2, 2), (3, 3), and (4, 4).

Bresenham's algorithm: special cases

Positive Slope Greater Than 1

When the slope of a line is greater than 1, it means the line is steep..

To handle this, we switch the roles of x and y. This essentially rotates the line by 90 degrees, making it flatter

Algorithm

1.Switch x and y: Treat the line as if it were from (y_1, x_1) to (y_2, x_2) .

2.Apply Bresenham's algorithm:

1. Calculate the decision parameter P.
2. Iterate, choosing between incrementing y or both x and y based on P.

3.Plot the points:

1. For each point (x, y) calculated, plot the point (y, x) on the original coordinate system.

Example 4 : Consider a line from (2, 1) to (4, 5).

Solution

The slope is $(5-1)/(4-2) = 2$, **which is greater than 1**

1. Identify the special case:

• $dy > dx$, so we need to interchange x and y.

2. Interchanged coordinates:

• New starting point: (1, 2)

• New ending point: (5, 4)

3. Calculate dx and dy:

• $dx = 5 - 1 = 4$

• $dy = 4 - 2 = 2$

4. Initialize the decision parameter p0:

• $p0 = 2 * dy - dx = 2 * 2 - 4 = 0$

5. Plot the first point (x1, y1):

• Plot the point (2, 1).

6. Iterate until x = x2:

$X=1, y=2, p0 \geq 0, \rightarrow \text{plot } (2, 3), p1 = p0 + 2 * (dy - dx) = -2$

$X=2, y=3, p1 < 0, \rightarrow \text{plot } (3, 3), p2 = p1 + 2 * dy = 2$

$X=3, y=3, p2 \geq 0, \rightarrow \text{plot } (4, 4), p3 = p2 + 2 * (dy - dx) = -2$

$X=4, y=4, p3 < 0, \rightarrow \text{plot } (5, 4), p4 = p3 + 2 * dy = 2$

7. Plot the final point (x2, y2):

• Plot the point (4, 5).

8. Map back to original coordinates:

• Plot the points: (2, 1), (3, 2), (3, 3), (4, 4), (4, 5).